

Payment Gateways

1. Basic Payment Questions

Q.1: Which payment gateways have you integrated?

A payment gateway is a service that securely processes online payments between customers, merchants, and banks.

- **Razorpay (Popular in India)**
- **Stripe (Global)**
- **Paypal**
- **Cashfree**
- **PayU**
- **PhonePe Payment Gateway**

Interviewer Answer:

I have worked with Razorpay and understand the integration flow. The overall process is similar across Stripe, Paypal, and Cashfree, with differences in APIs and features.

Q.2: Explain the complete payment flow from frontend to backend.

STEP 1: User Clicks "Pay Now" ----- >

STEP 2: React sends a request to the Node.js backend ---- > **POST /create-order**

STEP 3: Backend validates:

- **User Authentication**
- **Product Availability**
- **Price from the Database (not from frontend)**

STEP 4: Backend creates an order using the payment gateway --- >

```
const order = await razorpay.orders.create({amount: 50000, currency: "INR"});
```

STEP 5: Backend Returns -- >

- **Order ID**
- **Amount**
- **Currency**

To React --- >

STEP 6: React opens the payment gateway.

STEP 7: User completes payment.

STEP 8: Payment gateway returns

- **payment_id**

- **order_id**
- **signature**

STEP 9: React send these details to the backend --- > **POST /verify-payment**

STEP 10: Backend verifies the signature.

STEP 11: Backend updates MongoDB ---- > **Payment Status = Paid**

STEP 12: Backend returns success.

Q.3: Why should payment creation happen on the backend instead of the frontend?

Because the backend is trusted, while the frontend is controlled by the user. If order creation happened on the frontend, a user could manipulate:

- **Price**
- **Product ID**
- **Discount**
- **Quantity**

Example - > Actual Amount: 1000, User changes request to amount 10, If the backend trusts the frontend, the payment will be created for Rs. 10.

Correct Approach:

- **Fetch the product price from the database.**
- **Calculate the total on the backend.**
- **Create the order using the backend.**

Interviewer Answer:

Payment creation must always happen on the backend because secret keys, pricing logic, and order validation must remain secure.

Q.4: What is Payment Gateway?

A payment gateway is a secure service that collects payment details and connects your application with banks or payment processors.

It:

- **Encrypts payment information**
- **Processes payment requests**
- **Returns success or failure**
- **Ensure secure transactions**

Exmpale:

- **Razorpay**
- **Stripe'**
- **Paypal**

- **Cashfree**

Q.5: What is Payment Processor?

A payment processor is responsible for transferring money between the customer's bank and the merchant's bank.

It:

- **Communicate with banks**
- **Checks available funds**
- **Approves or declines transactions**
- **Settles funds**

The processor performs the actual financial transaction behind the scenes.

Q.6: What is the difference between Payment Gateway and Payment Processor?

Payment Gateway

- Collects payments information
- Connects your app to payment services
- Handles user interaction
- Returns payment status

Payment Processor

- Moves money between banks
- Communicates with banks
- Handles financial settlement
- Approves or declines the transaction

Note:

- **Payment Gateway** – Receptionist who accepts your request
- **Payment Processor** – Banker who actually transfers the money

Q.7: What is an Order Id?

An Order ID is a unique identifier generated by the payment gateway for each payment order.

Like: **order_Pv7A8J2Hd9Xyz**

It is used to:

- Identify the transaction
- Prevent duplicate payments
- Link the payment with your database
- Verify the payment later

Q.8: What is a Payment Intent (Stripe)?

A Payment Intent is a Stripe object that represents a customer's payment before it is completed.

It tracks the payment lifecycle, including:

- **Amount**
- **Currency**

- **Payment Method**
- **Current status**

Possible Statuses:

- **requires_payment_method**
- **requires_confirmation**
- **processing**
- **succeeded**
- **canceled**

It also supports authentication flows like 3D Secure and helps prevent duplicate charges.

Q.9: What is a Client Secret?

A Client Secret is a temporary token generated by Stripe for a specific Payment Intent.

It is sent from the backend to the frontend and allows the frontend to securely complete the payment.

Important:

- It is not Stripe Secret Key.
- It is safe to use on the frontend for that payment only.
- The Stripe secret key must always remain on the backend.

Q.10: What is the difference between Authorization and Capture?

- | | |
|---------------------------------|---|
| • Authorization | Capture |
| • Reserves the customer's funds | Actually transfers the funds |
| • No money is transferred yet | Money is deducted from the customer's account |
| • Can expire if not captured | Completes the transaction |

Example:

- You book a hotel room.
- The hotel first authorizes Rs. 5000 on your card to ensure funds are available.
- After your stay, the hotel captures the payment, and the amount is actually deducted.

Interviewer Answer:

Authorization checks and reserves the customer's available funds, while Capture finalizes the transaction and transfers the money to the merchant.

2. MERN Specific Questions

Q.11: How do you integrate Razorpay in a MERN application?

The integration involves both the React frontend and the Node.js backend.

STEP 1: Create a Razorpay account

- Get your Key ID and Key Secret.
- Store the Key Secret in the backend .env file.

STEP 2: Backend creates an order -- > The frontend requests the backend to create an order -- > POST /api/payment/create-order

The backend:

Validate the user.

- Fetches the final amount.
- Calculates the final amount.
- Creates the Razorpay order.
- Returns:
 - **order_id**
 - **amount**
 - **currency**

STEP 3: Frontend Opens Razorpay checkout

Using the returned order details, React opens the Razorpay payment popup.

STEP 4: User completes Payment

Razorpay returns:

- **razorpay_payment_id**
- **razorpay_order_id**
- **razorpay_signature**

STEP 5: Verify Payment

- The frontend sends these values to the backend -- > **POST /api/payment/verify**
- The Backend verifies the signature using HMAC SHA256.

STEP 6: Update Database

If verification succeeds:

- Save payment details.
- Update order status to Paid.
- Return success.

Interviewer Answer:

In Mern, the backend creates the Razoray order, the React frontend opens Razorpay Checkout, and after payment the backend verifies the signature before updating MongoDB.

Q.12: How do you integrate Stripe Checkout?

STEP 1: User clicks Checkout.

STEP 2: React sends a request to the backend -- > **POST /create-checkout-session**

STEP 3: Backend creates a Stripe Checkout Session.

Example:

```
const session = await stripe.checkout.sessions.create
({
  payment_method_types: ['card'],
  line_items: [...],
  mode: 'payment',
  success_url: '...',
  cancel_url: '...'
});
```

STEP 4: Backend returns the Checkout Session ID.

STEP 5: React redirects the user to Stripe Checkout.

STEP 6: User completes payments.

STEP 7: Stripe sends a webhook to the backend confirming the payment.

STEP 8: Backend updates the order status in mongoDB.

Interviewer Answer:

Stripe Checkout is integrated by creating a Checkout Session on the backend redirecting the user from React, and using webhooks to confirm and record successful payments.

Q.13: How do you generate an order on the backend?

The backend should generate the order because it is the trusted layer.

STEP:

- Authenticate the user.
- Validate the request.
- Fetch the product price from the database.
- Calculate the total amount.
- Create the payment gateway order.
- Return the order details to the frontend.

Example:

```
const order = await razorpay.orders.create
({
  amount: amount * 100,
  currency: 'INR',
  receipt: 'receipt_001'
});
```

Interviewer Answer:

The backend generates the order after validating the user and calculating the amount from the database, ensuring the client cannot manipulate the payment.

Q.14: How do you verify payment after success?

Never trust the frontend response alone.

After payment, the frontend sends:

- **payment_id**
- **order_id,**
- **signature**

The backend:

1. Creates the HMAC SHA256 hash using the order ID and payment ID with the secret key.
2. Compares the generated hash with the received signature.
3. If they match:
 - **Payment is valid.**
 - **Update MongoDB or Database.**
4. If they don't match:
 - **Reject the payment.**

Interviewer Answer:

Payment verification is done on the backend by validating the payment signature with the gateway's secret key. Only after successful verification do we mark the payment as paid.

Q.15: Why shouldn't API keys be exposed in React?

React code runs in the user's browser, so any embedded secret can be viewed.

If a secret key is exposed:

- Attackers can create fake payments.
- They can misuse your payment account.
- Financial loss can occur.

Only public keys (such as Razorpay KEY ID or Stripe Publishable Key) may be used on the frontend.

Secret keys must remain on the backend.

Interviewer Answer:

Frontend code is publicly accessible, so secret API keys must never be exposed. Only publishable keys should be used in React.

Q.16: Where should Secret, Keys be stored?

Secret keys should be stored in the backend environment variables and accessed using **process.env**. They should never be exposed in frontend code or committed to version control.

Q.17: Which environment variables are required for payment integration?

For Razorpay:

RAZORPAY_KEY_ID=

RAZORPAY_KEY_SECRET=

For Stripe:

STRIPE_SECRET_KEY=

STRIPE_PUBLISHABLE_KEY=

STRIPE_WEBHOOK_SECRET=

Interviewer Answer:

The required environment variables depend on the gateway, but typically included API Keys, webhook secret, frontend URL and server configuration.

Q.18: How do you call the backend before opening the payment gateway?

The frontend should never open the payment gateway directly.

Flow:

1. User clicks the Pay Now.
2. React sends a request to the backend --- > POST /create-order
3. **Backend:**
 - Authenticate the User.
 - Validate the Order.
 - Calculates the amount.
 - Creates the payment gateway order.
 - Returns the order details.
4. React receives the response.
5. React opens the payment gateway using the returned order_id, amount and other required information.

Interviewer Answer:

Before opening the payment gateway, React first calls the backend to create a secure payment order. Once the backend returns the order details, the frontend launches the payment gateway using those values. This ensures that pricing and order creation remain secure.

3. Security Questions

Q.19: How do you verify Razorpay payment signature?

Razorpay returns these values after payment:

- razorpay_order_id
- razorpay_payment_id
- razorpay_signature

The backend generates its own signature using the Order ID, Payment ID and Razorpay Secret Key.

generatedSignature = HMAC_SHA256(order_id + '|' + payment_id, RAZORPAY_KEY_SECRET)

If the generated signature matches the received signature, the payment is authentic.

Example:

```
import crypto from 'crypto';

const generatedSignature = crypto.createHmac('sha256',
process.env.RAZORPAY_KEY_SECRET).update(`${razorpay_order_id}|${razorpay_payment_id}`).digest('hex');
if (generatedSignature === razorpay_signature)
{
    // Payment Verified
} else {
    // Invalid Payment
}
```

Interviewer Answer:

I never trust the frontend response. After payment, the backend regenerates the HMAC SHA256 signature using the Order ID, Payment ID, and Secret Key.

If both signature match, the payment is genuine.

Q.20: Why is HMAC SHA256 used?

HMAC SHA256 is to ensure:

- Data Integrity
- Authentication
- Tamper Detection

If generated signature changes completely.

Without the secret key, attackers cannot generate a valid signature.

Interviewer Answer:

HMAC SHA256 guarantees that payment data hasn't been modified and confirms that the response actually came from **Razorpay**.

Q.21: What happens if signature verification fails?

If verification fails:

Do not mark payment as **successful**.

Return an error.

- Log the incident.
- Keep payment status as failed or pending.
- Ask the customer to retry if needed.

Never trust a payment without verification.

Interviewer Answer:

If signature verification fails, I immediately reject the payment because it could indicate tempering or a fake response.

Q.22: How do you prevent payment tampering?

Correct Approach:

- Calculate amount on the backend.
- Fetch product price from Database.
- Never trust frontend price.
- Verify payment signature.
- Use HTTPS.
- Validate JWT.
- Store Secret Keys only on backend.
- Use Webhooks.

Example:

`amount = req.body.amount; ---- > Wrong`

`amount = product.price; ----- > Correct`

Interviewer Answer:

Payment Tampering is prevented by calculating prices on the backend, verifying signatures, authenticating users, and never trusting client-provided payment details.

Q.23: How do you prevent duplicate payments?**Techniques:**

- Disable Pay button after first click.
- Use unique Order IDs.
- Use idempotency.
- Check database before saving payment.
- Ignore duplicate webhook events.

Example:

```
const payment = await Payment.findone(  
{  
  paymentId: razorpay_payment_id  
})
```

Interviewer Answer:

I prevent duplicate payments using unique order IDs, idempotent APIs, frontend button disabling, and duplicate payment checks in the database.

Q.24: Why should payment amount never come directly from the frontend?

Because frontend code can be modified.

Example:

Website Price = 1000, User changes request --> { "amount": 10}, if backend trusts this value: Customer pays 10 instead of 1000.

Correct Flow:

Frontend --> Product ID, Backend --> Fetch Product, Database --> Price = 1000, Create Order.

Interviewer Answer:

The backend should always calculate the payment using trusted database values because client-side can be manipulated.

Q.25: What is PCI DSS?

PCI DSS stands for: **Payment Card Industry Data Security Standard.**

It is an international security standard for businesses handling card payments.

Requirements include:

- Encrypt card data.
- Protect customer information.
- Restrict access.
- Use secure audits.
- Regular security audits.

If you use Stripe or Razorpay standard for protecting cardholder data. By using hosted payment pages from providers like Stripe or Razorpay, much of the PCI compliance burden is handled by the payment gateway.

Q.26: Should card details be stored in Database?

No,

Store only:

- Payment ID
- Order ID
- Amount
- Status
- User ID
- Gateway Name

Reason:

- PCI DSS violation
- Security Risk
- Legal Compliance issues

Interviewer Answer:

No, Card details should never be stored in Database. Only non-sensitive payment metadata should be stored.

Q.27: Why should HTTPS always be used?

HTTPS encrypts communication between:

Browser < ---- > Server

Benefits:

- Prevents Man-in-the-Middle attacks.

- Encrypts payment information.
- Protects authentication tokens.
- Prevents data interception.

Without HTTPS:

Attackers could intercept sensitive information.

Interviewer Answer:

HTTPS encrypts all communication between the client and server, protecting payment data and authentication tokens from interception.

4. Webhook Questions

Q.28: What is a Webhook?

A Webhook is an HTTP callback sent automatically by the payment gateway to your backend when a payment event occurs.

Instead of your application repeatedly asking for updates, the payment gateway pushes the event.

Example:

- Payment succeeded
- Payment Failed
- Refund processed

Interviewer Answer:

A Webhook is a server-to-server notification from the payment gateway that informs my backend whenever a payment event occurs.

Q.29: Why are Webhooks needed if payment success already comes on the frontend?

The frontend response is not reliable because:

- User may close the browser.
- Internet may disconnect.
- JS can be manipulated.

Webhooks are reliable because they come directly from the payment gateway.

Interviewer Answer:

Frontend success should never be final source of truth. I rely on webhooks because they are sent directly from the payment gateway to my backend.

Q.30: How do you verify webhook signatures?

The payment gateway sends:

- Webhook payload
- Signature in request headers

The backend:

- Computes an HMAC SHA256 hash of payload using the webhook secret.
- Compares it with the received signature.
- Processes the webhook only if they match.

Interviewer Answer:

I verify webhook signatures using the webhook secret. If the generated signature matches the received one, I process the event: otherwise, I reject it.

Q.31: What happens if the user closes the browser after payments?

The payment can still succeed.

Even if the frontend never receives the success response, the payment gateway sends a webhook.

The Backend:

- Receives the webhook.
- Update Database.
- Marks the order as Paid.

Interviewer Answer:

The payment is still recorded because the webhook reaches the backend even if the user closes the browser.

Q.32: Which payment status should be trusted?

Priority:

- Webhook
- Backend signature verification
- Frontend response

The frontend is only for improving the user experience, not for deciding payment success.

Interviewer Answer:

The webhook is the most reliable source because it comes directly from the payment gateway. Frontend responses should never be trusted as the final confirmation.

Q.33: How do you retry failed webhooks?

Most payment gateways automatically retry failed webhooks.

On your backend:

- Return HTTP 200 only after successful processing.
- If processing fails, return an error.
- Make the webhook endpoint idempotent so repeated events don't create duplicate records.

Interviewer Answer:

Payment gateways automatically retry failed webhooks. My webhook handler is idempotent so repeated deliveries don't cause duplicate updates.

Q.34: What webhook events are important?**Common events include:**

- Payment Success
- Payment Failed
- Refund Processed
- Refund Failed
- Charge Captured
- Subscription Created (if applicable)
- Subscription Cancelled (if applicable)

Interviewer Answer:

The most important webhook events are payment success, payment failure, refunds, and capture because they directly affect order and payment status.

5. Database Questions

Q.35: What payment fields do you store in MongoDB?**Typical Fields:**

```
{  
  userId,  
  orderId,
```

```
    paymentId,  
    gateway,  
    amount,  
    currency,  
    status,  
    transactionDate,  
    receipt,  
    createdAt,  
    updatedAt  
}
```

Never Store:

- Card Number
- CVV
- PIN

Interviewer Answer:

I store payment metadata such as order ID, payment ID, user ID, amount, currency, gateway, and payment status. Sensitive card details are never stored.

Q.36: Which payment statuses do you maintain?

Common statuses:

- Pending
- Processing
- Paid,
- Failed,
- Refunded,
- Cancelled (optional)

Flow:

- Pending --> Processing --> Paid.
- Pending --> Failed.
- Paid --> Refunded.

Interviewer Answer:

I generally maintain statuses like Pending, Processing, Paid, Failed, and Refunded to accurately track the payment lifecycle.

Q.37: How do you avoid duplicate payment records?

Methods:

- Store unique `paymentId`.
- Add a unique index on `paymentId`.
- Check before inserting
- Make webhook processing idempotent.

Example:

```
const exists = await Payment.findOne({ paymentId });
```

```
if ( ! exists )
```

```
{
```

```
    await Payment.create( paymentId );
```

```
}
```

Interviewer Answer:

I enforce a unique payment ID in the database and make webhook processing idempotent to prevent duplicate records.

Q.38: How do you link Payment and Order collections?

Each payment stores the related **Order ID**.

Example:

Order Collection -- >

```
{
```

```
  _id: ObjectId('123'),
```

```
  total: 1500,
```

```
  status: 'Pending'
```

```
}
```

Payment Collection -- >

```
{
```

```
  orderId: ObjectId('123'),
```

```
  paymentId: 'pay_xyz',
```

```
  amount: 1500,
```

```
  status: 'Paid'
```

}

When payment succeeds:

- Update the Payment document.
- Update the corresponding Order document to Paid.

Interviewer Answer:

I link the Payment collection to the Order collection using the Order ID. This allows me to retrieve payment details for an order and keep both collections synchronized.

Q.39: Should payment history be editable?

No,

Payment history is a financial audit trail and should remain immutable.

If something changes:

- Create a refund record.
- Create an adjustment entry.
- Log the change.

Never modify historical payment records.

Interviewer Answer:

No, Payment history never be edited because it serves as an audit trail. Any corrections should be recorded through refunds or adjustment records than modifying the original transaction.

6. Refund Questions

Q.40: How do refunds work?

A refund is the process of returning money to the customer after a successful payment.

Refund Flow:

- Customer requests a refund.
- Backend validates the request.
- Check if the payment exists and is eligible for a refund.
- Call the payment gateway's Refund API.
- Payment gateway processes the refund.
- Gateway sends a webhook (Refund/Failed).

- Backend updates the payment and order status in MongoDB.

Flow Diagram:

- **Customer -- > Request Refund -- > Backend -- > Validate Payment -- > Payment Gateway Refund API -- > Refund Processed -- > Webhook -- > MongoDB updated.**

Interviewer Answer:

A refund is initiated from the backend after validating the payment. The payment gateway processes the refund, sends a webhook, and then the backend updates the database.

Q.41: Who initiates the refund?

The merchant (your backend) initiates the refund by calling the payment gateway's Refund API.

Customers cannot directly refund themselves.

Typical Reason:

- Order cancelled
- Product out of stock
- Duplicate Payment
- Damaged Product
- Customer Return

Interviewer Answer:

Refunds are initiated by the merchant's backend after validating the business rules. The backend then calls the payment gateway's refund API.

Q.42: Can refunds be partial?

Yes.

Most payment gateways support:

- **Full Refund**
- **Partial Refund**

Example:

Payment Amount = 5000

Refunds:

- 1000

- 2000
- Remaining 2000 later

Some gateways allow multiple partial refunds until the full amount is refunded.

Interviewer Answer:

Yes. Most gateways support partial refunds, allowing only a portion of the original payment to be refunded.

Q.43: How do you track refund status?

Store refund information in the database.

Example:

```
{  
  refunded,  
  paymentId,  
  amount,  
  refundStatus,  
  refundedAt  
}
```

Possible statuses:

- Pending
- Processing
- Refunded
- Failed

The webhook updates the status after the gateway completes the refund.

Interviewer Answer:

I track refund status in MongoDB using fields like refund ID, payment ID, amount, and refund status, and update them through webhook events.

Q.44: What webhook events are fired for refunds?

Common refund webhook events:

- Refund Initiated
- Refund Processing
- Refund Successful
- Refund Failed

When the backend receives these events, it:

- Verifies the webhook signature.
- Updates the refund status.
- Updates the related payment/order if needed.
- Logs the event.

Interviewer Answer:

Refund webhooks notify the backend about refund progress. After verifying the webhook signature, I update the refund and payment status in the database.

7. Failure Handling

Q.45: What happens if payment succeeds but database update fails?

This is a common production scenario.

Suppose:

- Customer pays Rs. 1000
- Payment gateway confirms success
- MongoDB goes Down

If you only rely on the frontend, the payment record is lost.

Solution:

- Never trust only the frontend.
- Use webhooks.
- Retry database updates.
- Log failed operations.
- Use background jobs or message queues if required.

Interviewer Answer:

Even if the database update fails, the payment gateway will resend the webhook. My system retries the update until the payment is successfully recorded.

Q.46: What if payment fails after order creation?

Order creation and payment are separate steps.

Flow:

- **Order Created -- > Customer Pays -- > Payment Failed.**

The Backend Should:

- Keep the order status as Pending or Payment Failed.
- Allow the user to retry payment using the same order or create a new payment order, depending on business rules.

Do not mark the order as Paid.

Interviewer Answer:

If payment fails after order creation, I keep the order in a pending or failed state and allow the customer to retry payment.

Q.47: How do you retry failed payments?

There are 2 – common approaches:

Option 1:

- Create a new payment order and let the customer pay again.

Option 2:

- Reuse the existing order if the payment gateway supports it and the order hasn't expired.

The backend should:

- Verify the previous payment failed.
- Prevent duplicate successful payments.
- Update payment history.

Interviewer Answer:

For failed payments. I validate the previous attempt and then either create a new payment order or reuse the existing one if supported, ensuring duplicate payments are prevented.

Q.48: How do you prevent double-click payment?**Frontend:**

- Disable the Pay Now button immediately after the first click.
- Show a loading spinner.

Backend:

- Use unique order IDs.

- Check if a successful payment already exists.
- Implement idempotent APIs

Example:

```
<button disabled={loading}>  
  { loading ? 'Processing...' : 'Pay Now' }  
</button>
```

Interviewer Answer:

I prevent double payments by disabling the payment button on the frontend and validating duplicate payment attempts on the backend.

Q.49: How do you make payment APIs idempotent?

Idempotent means that making the same request multiple times produces the same result.

Example:

A user accidentally clicks Pay Now three times.

Without idempotency:

- Three payments could be created.

With idempotency:

- Only one payment is processed.
- The other return the existing result.

Implementation:

- Generate a unique idempotency key.
- Store it with the payment request.
- Reject or return the previous response if the same key is reused.

Interviewer Answer:

I make payment APIs idempotent by using unique idempotency key or unique order/payment IDs so repeated requests don't create duplicate transactions.

8. JWT & Authentication

Q.50: Should payment APIs require JWT authentication?

Yes, for authenticated users.

Reasons:

- Verify the user's identity.
- Prevent unauthorized payment requests.
- Associate payments with the correct account.
- Protect payment endpoints.

Expectations:

Guest checkout systems may not require JWT but should use other secure validation methods.

Interviewer Answer:

Yes, Payment APIs should generally require JWT authentication so that only authenticated users can initiate and manage their own payments.

Q.51: Can a guest user make a payment?

Yes, if business supports guest checkout.

In this case:

- No JWT is required.
- Collect customer details like email and phone number.
- Generate a temporary order.
- Validate payment securely on the backend.

Examples:

- Amazon Guest Checkout
- Many e-commerce websites

Interviewer Answer:

Yes. Guest users can make payments if the application supports guest checkout, but the backend must still validate the order and payment securely.

Q.52: How do you associate payment with the logged-in user?

After JWT verification, the backend extracts the user's ID from the token.

Example:

```
req.user = {
```



```
    id: 'user123'  
  };
```

When creating the payment:

```
const payment = await Payment.create( {  
  userId: req.user.id,  
  orderId,  
  paymentId,  
  amount,  
  status: 'Paid'  
});
```

This links the payment to the authenticated user and makes it easy to retrieve payment history.

Interviewer Answer:

After verifying the JWT, I extract the user's ID and store it with the payment record. This ensures every payment is linked to the correct authenticated user and supports secure payment history retrieval.

9. Production Questions

Q.53: How do you switch between Test Mode and Live Mode?

Payment gateways provide 2 – environment:

Test (Sandbox) Mode – Used during development and testing. No real money is transferred.

Live (Production) Mode – Used in production. Real transactions are processed.

The switch is usually done by changing environment variables, not by changing the code.

Example:

Development (.env)

- NODE_ENV=development
- RAZORPAY_KEY_ID=rzp_test_XXXXX
- RAZORPAY_KEY_SECRET=test_secret
- STRIPE_SECRET_KEY=sk_test_XXXXX

Production (.env)

- NODE_ENV=production
- RAZORPAY_KEY_ID=rzp_live_XXXXX
- RAZORPAY_KEY_SECRET=live_secret
- STRIPE_SECRET_KEY=sk_live_XXXXX

In Node.js:

```
const keyId = process.env.RAZORPAY_KEY_ID;  
const keySecret = process.env.RAZORPAY_KEY_SECRET;
```

Best Practices:

- Never hardcode API keys.
- Keep separate .env files for development and production.
- Never use test keys in production.
- Never expose secret keys to the frontend.

Interview Answer:

I switch between Test and Live mode by using different environment variables for each environment. The application code remains the same: only credentials change.

Q.54: How do you deploy payment integration securely?

Secure deployment involves protecting API keys, payment data, and backend endpoints.

Best Practice:

1. Store Secrets in Environment Variables:

- RAZORPAY_KEY_SECRET=XXXXXXXXX
- JWT_SECRET=XXXXXXXXX

Never commit these to GitHub.

2. Always Use HTTPS

All payment-related communication must be encrypted.

3. Verify Payment Signatures

Never trust the frontend response.

Always verify:

- Razorpay Signature
- Stripe Webhook Signature

4. Secure APIs

- JWT Authentication
- Input Validation
- Rate Limiting

5. Enables CORS Properly

Allow only trusted frontend domains

Bad -- > if origin = '*'

Good -- > if origin = '<https://mywebsite.com>'

6. Log Errors Only:

- Secret Keys
- Card Details
- JWT Tokens

Interviewer Answer:

For secure development, I store secrets in environment variables, enforce HTTPS, verify payment signatures, secure APIs with JWT, configure CORS correctly, and avoid logging sensitive information.

Q.55: How do you rotate Secret Keys?

Secret Key rotation means replacing old API keys with new ones without affecting the application.

STEPS:

1. Generate a new key in the payment gateway dashboard.
2. Update the server's environment variables.
3. Restart or redeploy the application.
4. Verify that new payments work correctly.
5. Disable the old key after confirming everything is working.

Why Rotate Keys:

- Prevent unauthorized access.
- Improve security.
- Meet compliance requirements.

Interviewer Answer:

I rotate secret keys by generating new credentials in the payment gateway, updating environment variables, redeploying the application, testing the integration, and then disabling the old keys.

Q.56: What should never be logged in production?

Never log:

- Secret API Keys
- JWT Tokens
- Card Number
- CVV
- PIN
- OTP
- Password
- Full webhook payload containing sensitive information
- Database credentials

Safe to log:

- Payment ID
- Order ID
- Status
- Timestamp
- Error Message (without sensitive data)

Example:

Wrong -- > `console.log(process.env.RAZORPAY_KEY_SECRET);`

Wrong -- > `console.log(req.body.cardNumber);`

Correct -- > `console.log({ paymentId, orderId, status });`

Interviewer Answer:

Sensitive information like secret keys, JWTs, card details, passwords, and OTPs should never be logged. I only log non-sensitive metadata such as payment IDs, order IDs, statuses, and error messages.

Q.57: How do you monitor failed payments?

A production application should continuously track failed payment attempts.

1. Methods

Store Failed Payments

```
{  
  paymentId,
```

```
    userId,  
    amount,  
    status: "Failed",  
    reason: "Insufficient Balance"  
}
```

2. Maintain Logs

Store:

- Error message
- Timestamp
- User ID
- Payment Gateway Response

3. Dashboard

Create an Admin Dashboard showing:

- Total Payments
- Successful Payments
- Failed Payments
- Refunds
- Pending Payments

4. Alerts

Send notification for:

- High failure rates
- Payment gateway downtime
- Repeated webhook failures

5. Retry Support

Allow customers to retry failed payments without creating duplicate successful transactions.

Interviewer Answer:

I monitor failed payments by storing failure records in the database, logging gateway responses, displaying analytics on an admin dashboard, setting up alerts for abnormal failure rates, and allowing secure payment retries.

10. Coding Questions

Q.58: Write an API to create an order.

- Authenticate the user.

- Validate the request.
- Fetch product price from the gateway.
- Create an order using the payment gateway.
- Return orderId, amount, and currency.

Q.59: Verify Razorpay payment signature.

Receive **orderId**, **paymentId**, and **signature**.

- Generate an HMAC SHA256 signature using the secret key.
- Compare it with the received signature.
- If they match, update the payment status to Paid.

Q.60: Implement Stripe Checkout session.

- Create a Checkout Session on the backend.
- Return the session ID to the frontend.
- Confirm payment through a webhook.

Q.61: Save payment in MongoDB.

Store only payment metadata, for example:

- User ID
- Order ID
- Payment ID
- Amount
- Currency
- Status
- Gateway
- Created At
- Never store card numbers or CVV.

Q.62: Create payment history API.

- Authenticate the user.
- Query payments using userId.
- Support pagination and sorting.
- Return only that user's payment history.

Q.63: Implement refund API.

- Validate the payment.
- Check refund eligibility.
- Call the payment gateway's Refund API.

- Save the refund request.
- Update status via webhook.

Q.64: Handle webhook endpoint.

- Verify the webhook signature.
- Process the event.
- Update the database.
- Return HTTP 200 after successful processing.
- Make the endpoint idempotent.

Q.65: Protect payment routes using JWT.

- Verify the JWT token.
- Extract the user ID.
- Allow only authenticated users to access payment APIs.
- Reject unauthorized requests with 401 Unauthorized.

Q.66: Update order status after successful payment.

- After successful signature verification or webhook.
- Update the payment collection to Paid.
- Update the Order collection to Paid.
- Record the payment timestamp.

Q.67: Handle duplicate webhook events.

- Check whether the event or payment has already been processed.
- Ignore duplicate events.
- Use unique event/payment IDs and idempotent logic so repeated webhook deliveries do not create duplicate records.

11. Scenario Based Questions

Q.68: A payment succeeded but the user closed the browser. How will your system update the order?

This is one of the most common production scenarios.

Scenario

- User clicks Pay Now.
- Payment is completed successfully.

- Before React receives the success response, the user closes the browser.
- The frontend never call /verify-payment.

Does the payment get lost?

No.

The Payment gateway (**Razorpay**/**Stripe**) sends a Webhook directly to the backend.

Flow

User --> Pay Now --> Payment Success --> Browser Closed --> Payment Gateway --> Webhook --> Backend --> Verify Signature --> Update MongoDB --> order = Paid.

Best Practice:

- Never depend only on the frontend.
- Always use webhooks.
- Verify the webhook signature.
- Update the database from the backend.

Interviewer Answer:

If the user closes the browser after payment, the frontend may never receive the success response. However, the payment gateway sends a webhook to my backend. After verifying the webhook signature, I update the payment and order status in MongoDB, ensuring the transaction is not lost.

Q.69: the frontend reports success, but the webhook reports failure. Which do you trust and why?

Always trust the webhook.

Why?

The frontend can be:

- Manipulated.
- Interrupted.
- Closed.
- Modified by users.

The Webhook is sent directly from the payment gateway to your backend.

Priority:

Webhook -- > Backend Signature Verification -- > Frontend Response.

Best Practice:

If -- > Frontend: **success.**

Webhook: **Failure.**

Then -- > Order = **Failed.**

Because the webhook is the final source of truth.

Interviewer Answer:

I always trust the webhook because it comes directly from the payment gateway. The frontend response only for improving the user experience and should never be considered the final confirmation of payment.

Q.70: A user refreshes the payment page and pays twice. How do you prevent duplicate orders?

This is common production issue.

Prevention Steps:

1. Unique order ID

- Every order should have unique ID.

2. Disable the Pay Button

```
<button disabled={loading}>
```

Pay Now

```
</button>
```

3. If found: Don't create another payment.

Idempotency

Use an idempotency key so repeated requests return the same result instead of creating multiple payments.

Interviewer Answer:

To prevent duplicate orders, I use unique order IDs, disable the Pay button after the first click, check for existing successful payments in the database, and implement idempotent payment APIs.

Q.71: Your webhook is called multiple times. How do you avoid duplicate database updates?

Most payment gateways retry webhooks until they receive a successful response.

Example:

- Webhook #1
- Webhook #2
- Webhook #3

All contain the same event.

Without protection: **Payment Saved 3 Times** ❌

Solution:

- Use idempotency:

Example:

```
const payment = await Payment .create({
    paymentId
});
if (payment) {
    return;
}
```

Or use: **eventId**

Store processed webhook event IDs.

If already processed: Ignore

Interviewer Answer:

I make the webhook handler idempotent by checking whether the payment or event has already been processed. If it has, I ignore the duplicate webhook.

Q.72: How would you design a scalable payment service in a large MERN application?

Use a layered architecture.

React --> Payment API --> Controller --> Payment Service --> Gateway Adapter --> Repository --> MongoDB

Recommended Folder Structure:

```
payment/
|
```

```
└─ controller
└─ service
└─ repository
└─ routes
└─ models
└─ middleware
└─ gateway
  └─ razorpay.js
  └─ stripe.js
  └─ paypal.js
└─
└─ webhook
└─ validators
└─ utils
```

Features:

- JWT Authentication
- Repository Pattern
- Service Layer
- Global Error Handler
- Webhooks
- Retry Mechanism
- Logging
- Rate Limiting
- Background Jobs
- Idempotency

Interviewer Answer:

In a large MERN application, I separate responsibilities into controllers, services, repositories, and gateway adaptors. This makes the payment module modular, scalable, and easy to extend with additional payment providers.

Q.73: How would you support multiple gateways (Stripe, Razorpay, Paypal) using common architecture?

Use the **Strategy Design Pattern**.

Instead of writing payment logic everywhere, create a common interface.

Example:

PaymentService

```
└─ RazorpayService
└─ StripeService
└─ PayPalService
```

Common Methods:

- createOrder()
- verifyOrder()
- refund()
- webhook()

Then select the gateway dynamically.

```
switch(gateway) {
  case "stripe":
  case "razorpay":
  case "paypal"
}
```

Benefits:

- Easy to add a new gateway.
- Cleaner code.
- Better Maintainability.
- Less duplication.

Interviewer Answer:

I would use the Strategy Pattern, where each payment gateway implements the same interface for opportunities like **createOrder**, **verifyPayment**, **refund**, and **webhook handling**. This allows switching or adding gateways with minimal changes.

Q.74: How do you ensure payment consistency in a distributed system?

In distributed system, different services (Payment, Order, Inventory, Notification) communicate asynchronously.

Best Practices:

- Use unique transaction IDs.

- Make APIs idempotent.
- Use message queues (RabbitMQ, Kafka, etc) for reliable event processing.
- Process webhooks asynchronously.
- Retry failed operations.
- Log every transaction.
- Use eventual consistency where immediate when possible: coordinate service through events.

Flow:

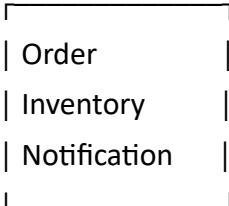
Payment Success

|



Message Queue

|



Interviewer Answer:

I ensure consistency using idempotent APIs, unique transaction IDs, reliable messaging, retries, logging, and event-driven communication so that every service eventually reaches the correct state.

Q.75: How would you handle payment timeout and abandoned transactions?

Sometimes:

- User closes the tab.
- Internet disconnects.
- Payment session expires.
- Customer leaves without completing payment.

These create Pending transaction.

Solution

1. Create the order with status Pending.
2. Wait for:
 - Successful payment.
 - Failed payment.
 - Timeout.

3. Use webhooks to update the final status.
4. Run a scheduled job (Cron Job) to find expired pending payments.
5. Mark expired orders as Cancelled or Expired base on business rules.
6. Release reserved inventory if applicable.

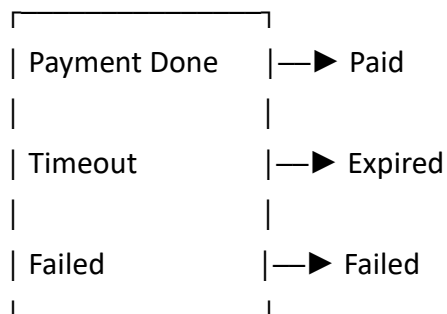
Example Flow:

Order Created

|

Pending

|



Interviewer Answer:

I create orders in a Pending state, rely on webhooks for payment updates, and use scheduled jobs to identify and expire abandoned transactions. If inventory was reserved, I release it after the order expires.